

State Handling & Auth

IF3110 – Web-based Application Development
School of Electrical Engineering and Informatics
Institut Teknologi Bandung

Task in Web Application

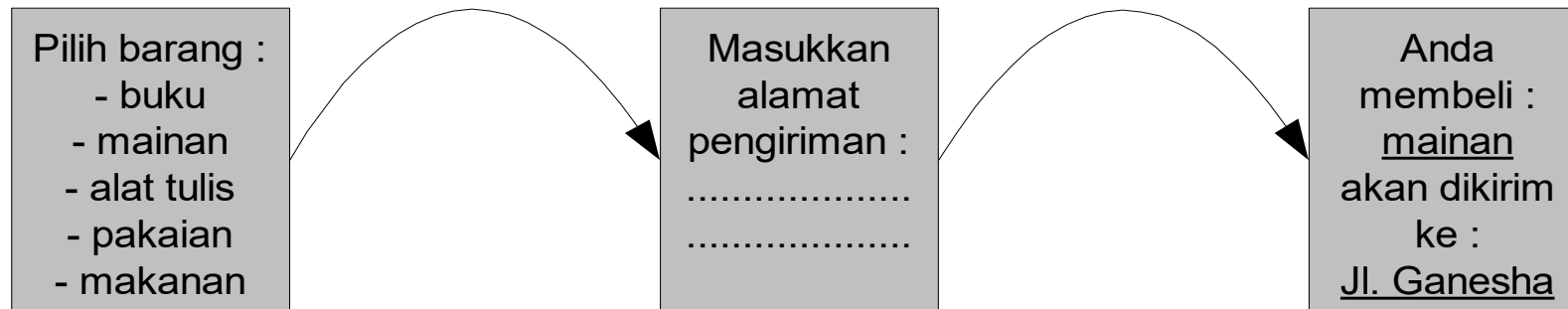
Application's features allow users to perform various tasks

To perform a task might involve one or more pages (as interface)

Example

1 page: read an article, add entry in a guestbook

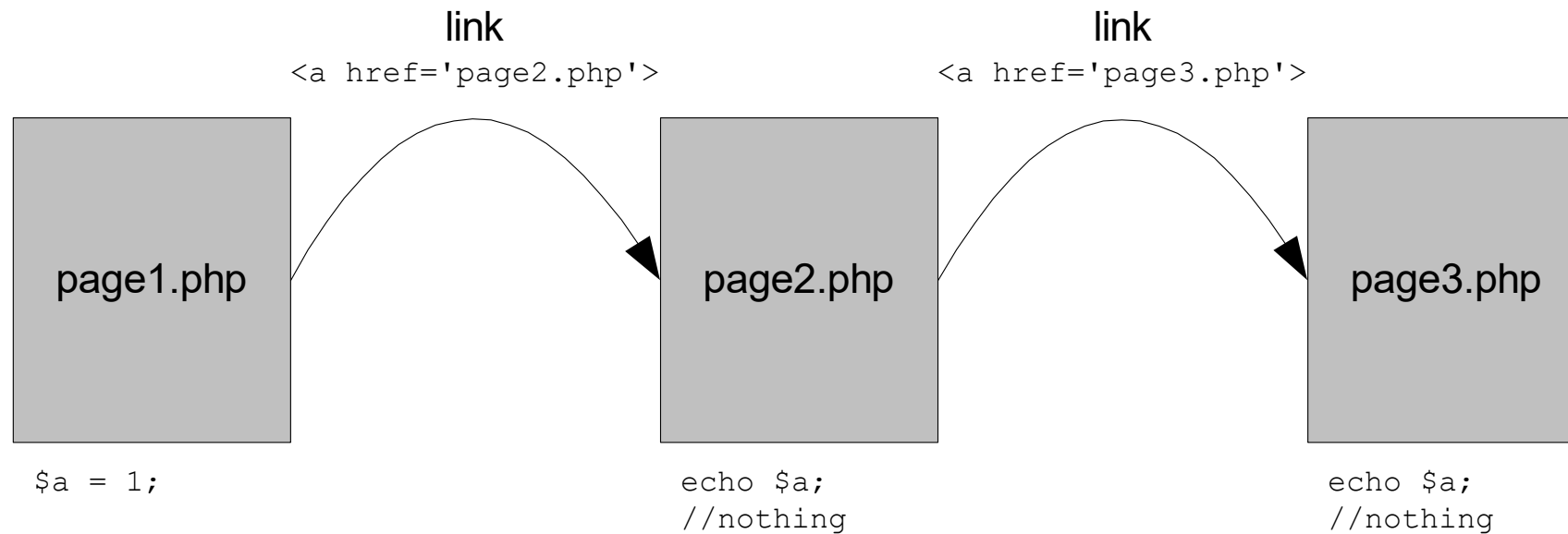
more pages: buying things online



Between pages should have a mechanism to manage the *state* of the task

Stateless HTTP

HTTP is *stateless*; no *state* is preserved between requests (at protocol level)



State Management

- **Definition** : a process to maintain state and page information over multiple requests for the same/different pages
- Web applications need to manage state, and that state can be maintained on the **client side**, on the **server side**, or through a **combination of both**. (Most cases is to use combination of both)

Common Techniques

Technique to overcome this limitation

- Message Passing
 - URL/form
 - Hidden field
- Browser Memory
- Browser Storage
 - Cookie
 - Local Storage/Session Storage
- Server Side
 - Session
 - Database
 - File (In Server)

Server-Side Authority - Treating the client (browser) as a hostile environment. The browser only holds references (tokens, IDs, hidden fields), while the **server holds the truth**

URL : Carrying state with each request

***Example a Academic Portal website**



```
GET /courses?department=STEI&page=2
```

The client sends the values again whenever they are needed. Keep passwords and session IDs out of URLs (will talk about session later)

URL and query parameters in Go

```
func products(w http.ResponseWriter, r *http.Request) {  
    q := r.URL.Query()  
    category := q.Get("department")  
    page := q.Get("page")  
  
    w.Header().Set("Content-Type", "text/plain; charset=utf-8")  
    fmt.Fprintln(w, category, page)  
}
```

/courses?department=STEI&page=2

URL state survives refresh

```
const params = new URLSearchParams ({  
  department: "STEI",  
  page: "2"  
});  
location.href = "/courses?" + params.toString();
```

Refresh requests the same query again.

A bookmark or shared link can preserve filters and pagination.

Useful for navigation state.

HTML forms carry named values

```
<form action="/register" method="post">
  <label>Username
    <input name="username" required>
  </label>
  <button type="submit">Register</button>
</form>
```

POST /register

Content-Type: application/x-www-form-urlencoded

username=Arya

Reading form data in Go

```
r.Body = http.MaxBytesReader(w, r.Body, 4096) //MUST limit the size
if err := r.ParseForm(); err != nil {
    http.Error(w, "Invalid form", http.StatusBadRequest)
    return
}
username := r.PostForm.Get("username")
if username == "" {
    http.Error(w, "Username required", http.StatusBadRequest)
    return
}
```

Parsing produces strings. The application still validates them.

Hidden fields

```
<form action="/enroll" method="post">  
  <input type="hidden" name="offering_id" value="123">  
  <button type="submit">Take Course</button>  
</form>
```

```
// After checking r.ParseForm()  
productID := r.PostForm.Get("offering_id") //MUST verify on server
```

Hidden describes the normal page UI, not a security boundary. Users can modify it via DevTools.

Browser Memory

A variable remembers the selection while the page runs.

```
let selectedCourseIds = [];
```

```
function render() {  
    document.querySelector("#count").textContent = String(  
        selectedCourseIds.length,  
    );  
}
```

Lost on page reload

Memory Store

A store is useful when the coordination work becomes harder than using a few ordinary variables. It is not compulsory for every page.

A shared store lets them:

- **Read** the same state.
- **Update** it through actions.
- **Subscribe** to changes so the page can update its display.

Examples: Zustand, Redux, MobX.

Zustand: Define the shared store

Keep the selected IDs and their update action together.

```
import { createStore } from "zustand/vanilla";

const store = createStore((set) => ({
  selectedCourseIds: [],
  addCourse: (id) =>
    set((state) => ({
      selectedCourseIds: state.selectedCourseIds.includes(id)
        ? state.selectedCourseIds
        : [...state.selectedCourseIds, id],
    })),
}));
```

addCourse adds an ID only if it is not already selected.

Zustand: connect the store to the page

Use the store from the previous slide.

```
function render(state) {  
  
  document.querySelector("#count").textContent = String(  
    state.selectedCourseIds.length,  
  );  
}  
  
document.querySelector("#add42")  
  .addEventListener("click", () => {  
    store.getState().addCourse(42);  
  });  
  
store.subscribe(render);  
render(store.getState());
```

On reload, all state in memory is gone.

1. Add course 42: count becomes **1**.
2. Reload: the page restarts and memory resets.
3. The count returns to **0**.

The shared store keeps the page consistent, but it persists nothing. Persistence requires **saving and restoring**.

localStorage for a theme preference

```
/* style.css */
body.dark { background: #16202b; color: white; }

// app.js
localStorage.setItem("theme", "dark");
const theme = localStorage.getItem("theme");
document.body.classList.toggle("dark", theme === "dark");
// localStorage.removeItem("theme");
```

A reload creates fresh JS memory, then code reads the saved preference.

Saving a Selected Course as JSON

```
localStorage.setItem(  
    "selectedCourses",  
    JSON.stringify(selectedCourses)  
);  
  
try {  
    const saved = JSON.parse(  
        localStorage.getItem("selectedCourses") ?? "[]"  
    );  
  
    selectedCourses = Array.isArray(saved) ? saved : [];  
} catch {  
    selectedCourses = [];  
}
```

Storage writes may fail. Keep the interface usable if persistence is unavailable.

Browser storage lifetimes

Mechanism	Reload	Usual lifetime
JS variable	Not Retained	Current document
sessionStorage	Retained	Tab's page session
localStorage	Retained	Until cleared / evicted

Web Storage belongs to an origin. It does not automatically travel in requests.

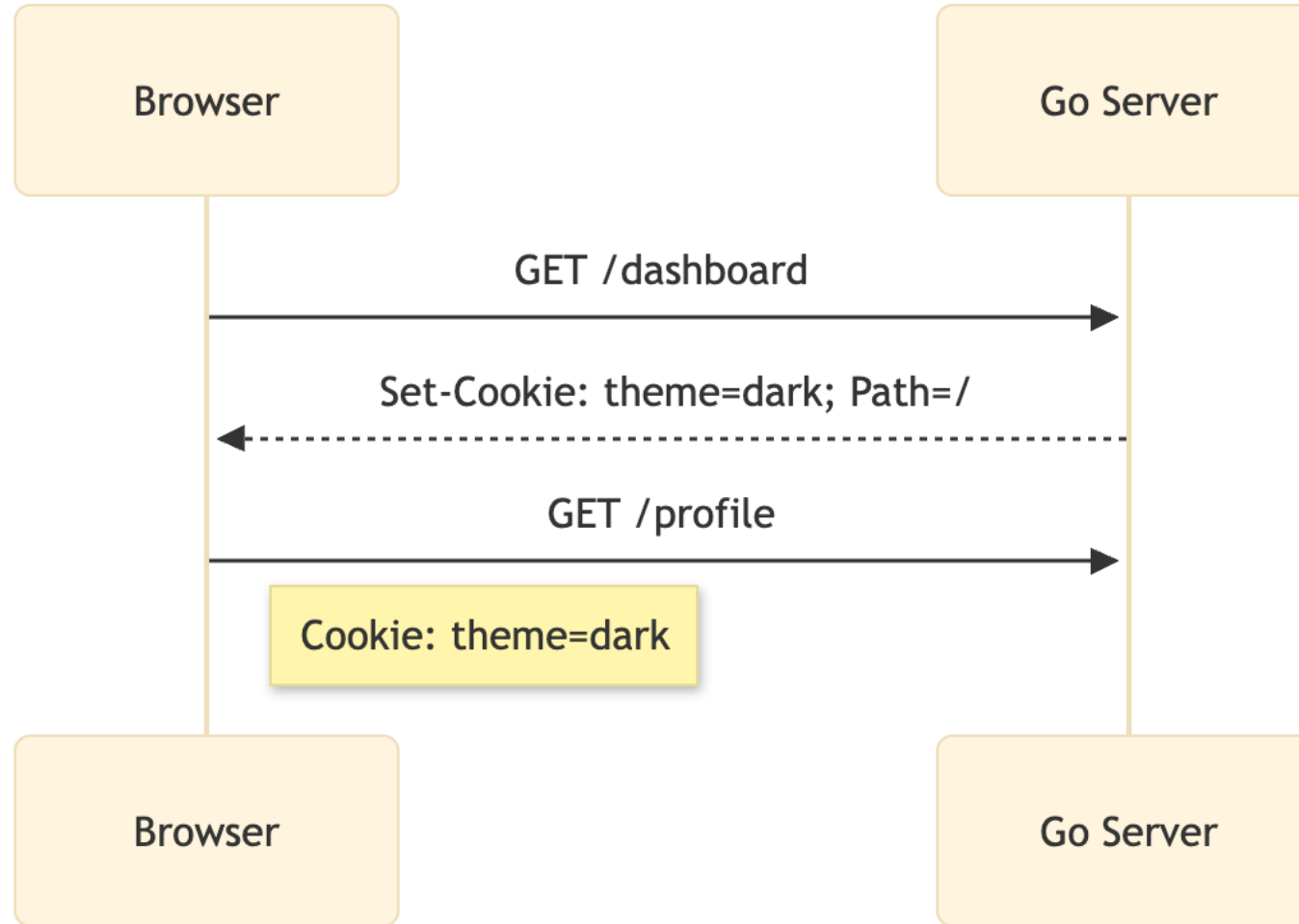
Sending browser state explicitly

```
const theme = localStorage.getItem("theme");

await fetch("/api/preferences", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ theme })
});
```

JavaScript copies the value into a request body when the server needs it.

Cookies connect browser storage to HTTP



Setting a cookie in Go

```
http.SetCookie(w, &http.Cookie{
    Name:      "theme",
    Value:     "dark",
    Path:      "/",
    MaxAge:    60 * 60 * 24 * 30,
    Secure:    true,
    SameSite: http.SameSiteLaxMode,
})
fmt.Fprintln(w, "Preference saved")
```

Set the cookie before writing the response. This example assumes HTTPS.

Reading and deleting cookies in Go

```
theme := "light"
if c, err := r.Cookie("theme"); err == nil && c.Value == "dark" {
    theme = "dark"
}

// To delete the cookie in a later response:
http.SetCookie(w, &http.Cookie{
    Name: "theme", Value: "", Path: "/", MaxAge: -1,
    Secure: true, SameSite: http.SameSiteLaxMode,
})
```

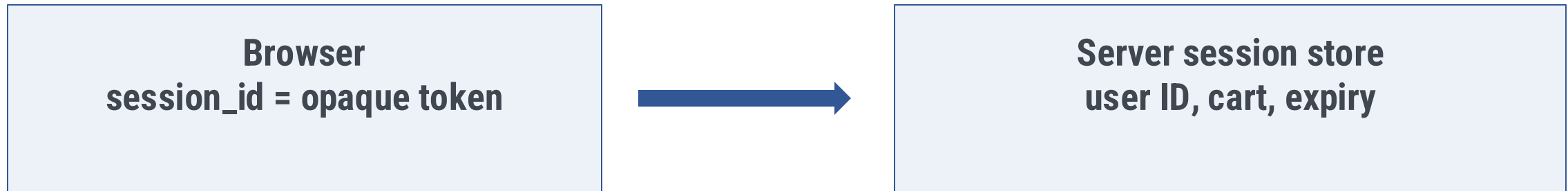
Handle missing cookies. A client can remove or change cookie values.

Cookie scope and security attributes

Attribute	Effect
Secure	Send over HTTPS
HttpOnly	Block document.cookie access
SameSite	Restrict sending in cross-site contexts
Path / Domain	Control which requests can receive the cookie
Max-Age / Expires	Set persistence and expiry in the browser

Keep cookies small. Browser privacy rules may block third-party cookies.

Server-side Session



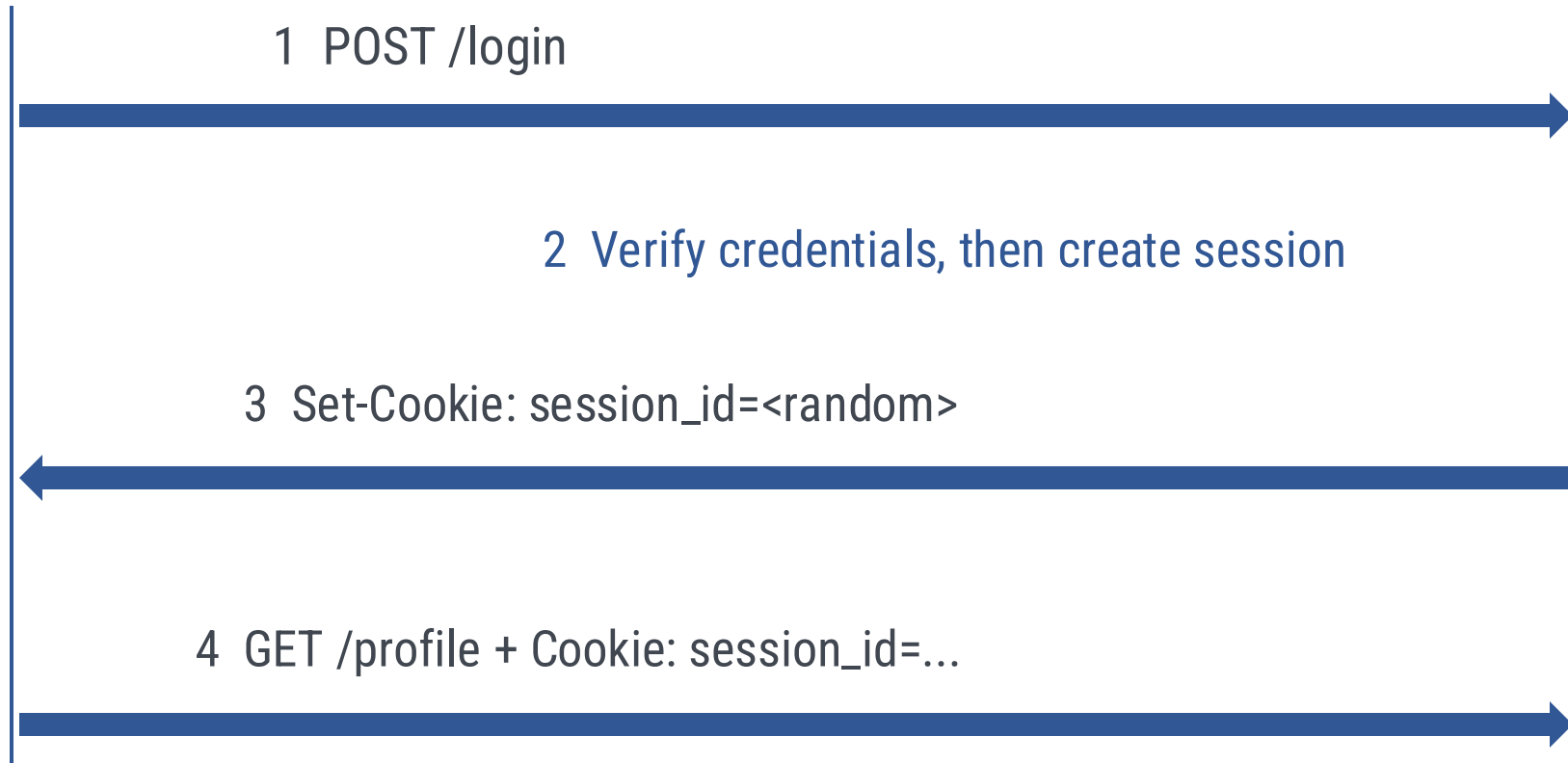
The browser presents an identifier.
The server looks up the corresponding state.

The session data stays on the server. The identifier still needs transport.

Session ID in a cookie

Browser

Go server + session store



A small Go session store

```
type Session struct {  
    UserID  string  
    Expires time.Time  
}  
  
var (  
    mu      sync.Mutex  
    sessions = map[string]Session{  
    })
```

An in-memory map suits a teaching demo. Multiple servers need a shared store.

Creating a session after login

```
var token [32]byte
if _, err := rand.Read(token[:]); err != nil {
    http.Error(w, "Session error", http.StatusInternalServerError)
    return
}

sid := hex.EncodeToString(token[:])
mu.Lock()
sessions[sid] = Session{
    UserID: userID, Expires: time.Now().Add(30 * time.Minute),
}

mu.Unlock()
```

Use unpredictable IDs. Never hard-code a shared session ID.

Sending and resolving the session ID

```
// Response after login
http.SetCookie(w, &http.Cookie{
    Name: "session_id", Value: sid, Path: "/",
    HttpOnly: true, Secure: true,
    SameSite: http.SameSiteLaxMode, MaxAge: 1800,
})
```

```
// A later request
c, err := r.Cookie("session_id")
if err != nil {
    http.Error(w, "Sign in", http.StatusUnauthorized)
    return
}
```

Checking the session and its lifetime

```
mu.Lock()
sess, ok := sessions[c.Value]
mu.Unlock()

if !ok || !time.Now().Before(sess.Expires) {
    http.Error(w, "Sign in", http.StatusUnauthorized)
    return
}
// Use sess.UserID, then authorize the requested action.
```

Browser cookie expiry and server session expiry are independent checks.

Session Limitation

Using session is simple, but it has limitations:

(in current class setup)

- **Data lives in one server** – another server cannot access the same session data.
- **Data may disappear on restart** – in-memory sessions are lost when the application stops.

(in general)

- **Hard to scale** – multiple application instances need a shared session store.
- **Limited persistence** – session data is not designed for long-term application data.
- **Concurrency and synchronization** can become more complex as the application grows.

Database

Database

A **database** is a system used to store, organize, and retrieve data reliably.

Instead of keeping data only in application memory, we can store it in a database so that it can:

- **Persist** even when the application restarts
- **Store large amounts of data**
- **Be shared** across multiple application instances
- **Search and update data efficiently**
- **Keep data structured and consistent**

Opening Database in Go

database/sql provides the interface; a driver talks to PostgreSQL.

- **sql.Open**: creates a handle; may not connect yet.
- **PingContext**: checks connectivity.
- Create one shared handle at startup; close it at shutdown.

Opening Database in Go

```
import (  
    "context"  
    "database/sql"  
    "time"  
    _ "github.com/jackc/pgx/v5/stdlib"  
)  
  
func openDB(ctx context.Context, dsn string) (*sql.DB, error) {  
    db, err := sql.Open("pgx", dsn)  
    if err != nil { return nil, err }  
  
    pingCtx, cancel := context.WithTimeout(ctx, 3*time.Second)  
    defer cancel()  
    if err := db.PingContext(pingCtx); err != nil {  
        db.Close()  
        return nil, err  
    }  
    return db, nil  
}
```

Querying Data From Database

Query by ID and distinguish a missing record from a database failure.

```
var title string
err := db.QueryRowContext(ctx,
    "SELECT title FROM course_offerings WHERE id = $1",
    offeringID,
).Scan(&title)

if errors.Is(err, sql.ErrNoRows) {
    return "", errOfferingNotFound
}
if err != nil { return "", err }
return title, nil
```

Scan copies the returned column into the Go variable.

Prepared Statements

A **prepared statement** lets us define a SQL query once and execute it multiple times with different values. Example:

```
db.QueryRowContext(ctx,  
    "SELECT title FROM course_offerings WHERE  
id = $1", offeringID)
```

Why use prepared statements?

- **Safer** – helps prevent SQL injection
- **Reusable** – execute the same query with different values
- **Cleaner** – SQL and user input stay separate
- **Potentially more efficient** for frequently repeated queries

Querying Multiple Rows

```
rows, err := db.QueryContext(ctx,
    `SELECT id, title FROM course_offerings`,
)
if err != nil {
    return err
}
defer rows.Close()

var offerings []CourseOffering

for rows.Next() {
    var o CourseOffering

    if err := rows.Scan(&o.ID, &o.Title); err != nil {
        return err
    }

    offerings = append(offerings, o)
}

if err := rows.Err(); err != nil {
    return err
}
```

Inserting Data

Use `INSERT` to create a new record.

```
_, err := db.ExecContext(ctx,
    `INSERT INTO enrollments (student_id, offering_id)
    VALUES ($1, $2)`,
    studentID, offeringID,
)

if err != nil {
    return err
}
```

Updating Data

```
result, err := db.ExecContext(ctx,
    `UPDATE course_offerings SET capacity = $1 WHERE id = $2`, capacity, offeringID)
if err != nil {
    return err
}

rows, err := result.RowsAffected() // tells us whether a record was actually updated.
if err != nil {
    return err
}

if rows == 0 {
    return errOfferingNotFound
}
```

NOTE: Always use WHERE to control which records are changed.

Delete Data

```
result, err := db.ExecContext(ctx,
    `DELETE FROM enrollments WHERE student_id = $1 AND offering_id = $2`,
    studentID, offeringID,
)

if err != nil {
    return err
}

rows, err := result.RowsAffected()
if err != nil {
    return err
}

if rows == 0 {
    return errEnrollmentNotFound
}
```

NOTE: Always use WHERE to control which records are deleted.

Transaction

```
tx, err := db.BeginTx(ctx, nil)
if err != nil {
    return err
}
defer tx.Rollback()

var stock int
err = tx.QueryRowContext(ctx,
    `SELECT stock FROM products WHERE id = $1 FOR UPDATE`, productID).Scan(&stock)
if err != nil {
    return err
}

if stock <= 0 {
    return errors.New("Out of Stock")
}

_, err = tx.ExecContext(ctx,
    `UPDATE products SET stock = stock - 1 WHERE id = $1`, productID)
if err != nil {
    return err
}

return tx.Commit()
```

Connection Pooling

Opening a new database connection for every request is expensive. A **connection pool** keeps a set of database connections ready to be reused.

Setting	Purpose
SetMaxOpenConns	Limit open connections
SetMaxIdleConns	Keep idle connections ready
SetConnMaxLifetime	Retire by age
SetConnMaxIdleTime	Retire by idle time
DB.Stats	Observe usage and waiting

sql.DB manages a pool of reusable connections shared by handlers.

ORM

An object-relational mapper (ORM) maps Go structs and operations to database records.

```
type Registration struct {  
    ID          int64  
    StudentID   int64  
    OfferingID  int64  
}
```

```
registration := Registration{  
    StudentID: studentID, OfferingID: offeringID,  
}  
err := gormDB.WithContext(ctx).Create(&registration).Error
```

GORM generates an INSERT. You still need **constraints, permission checks, and a transaction**.

Watch for **N+1 queries**: one list query, then another query per row.

Notes

Often you only have a single instance of SQL DBMS

- Chocking point to the Web App Performance, Single Point of Failure

Be careful with SQL wildcard

- Lead to slow query
- Unnecessary big result sets

Never trust other inputs

- Sanitize before store in Database
- Sanitize/escape before printing to client (after retrieval from database)

Open and Close Database Connection wisely

File

Things commonly done

- Reading a File
- Writing a File
- Working with CSV

Open a File

```
import (  
    "encoding/csv"  
    "fmt"  
    "io"  
    "os"  
)  
  
func main() {  
    file, err := os.Open("courses.csv")  
    if err != nil {  
        return err  
    }  
  
    defer file.Close()
```

Read CSV File

```
reader := csv.NewReader(file) // Create CSV Reader

for {
    row, err := reader.Read() // Read One Record

    if err == io.EOF {
        break
    }
    if err != nil {
        return err
    }

    fmt.Println(row)
}
```

Note: We can also use `reader.ReadAll()` to retrieve all lines

Write CSV File

```
func main() {  
    file, err := os.Create("courses.csv")  
    if err != nil {  
        return err  
    }  
    defer file.Close()  
  
    writer := csv.NewWriter(file)  
    defer writer.Flush() // Write buffered data to the file  
  
    writer.Write([]string{"id", "title", "credits"})  
    writer.Write([]string{"101", "Web Programming", "3"})  
    writer.Write([]string{"102", "Database Systems", "4"})  
}
```

Notes

- **Handle file errors correctly** (open, permission, space, etc.)

```
file, err := os.Create("courses.csv")
if err != nil {
    return nil, fmt.Errorf("failed to create courses.csv: %w", err)
}

defer file.Close() // Don't forget close the file
```

- **Sanitized supplied filenames**

```
user := r.FormValue("user")

// Allow only safe characters
validUser := regexp.MustCompile(`^[a-zA-Z0-9_-]+$`)
if !validUser.MatchString(user) {
    http.Error(w, "invalid user", http.StatusBadRequest)
    return
}

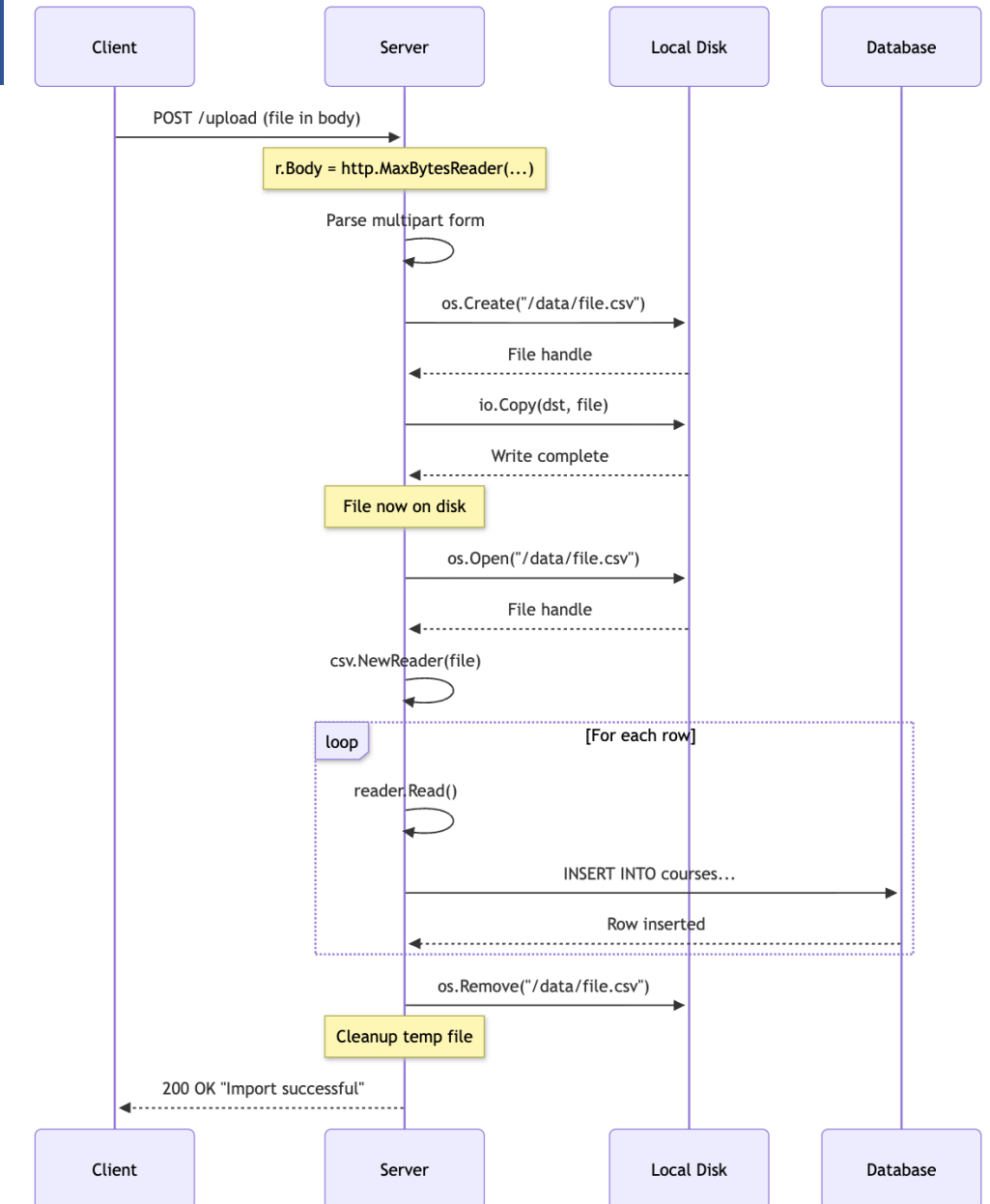
path := filepath.Join("/usr/local/data", user)
data, err := os.ReadFile(path)
```

File Handling in Web App

1. Upload the file to Server
2. Store the file in the Server
3. Open the file in the Server
4. Parse the file and Insert to Database
5. Clean up the temp file (if any)

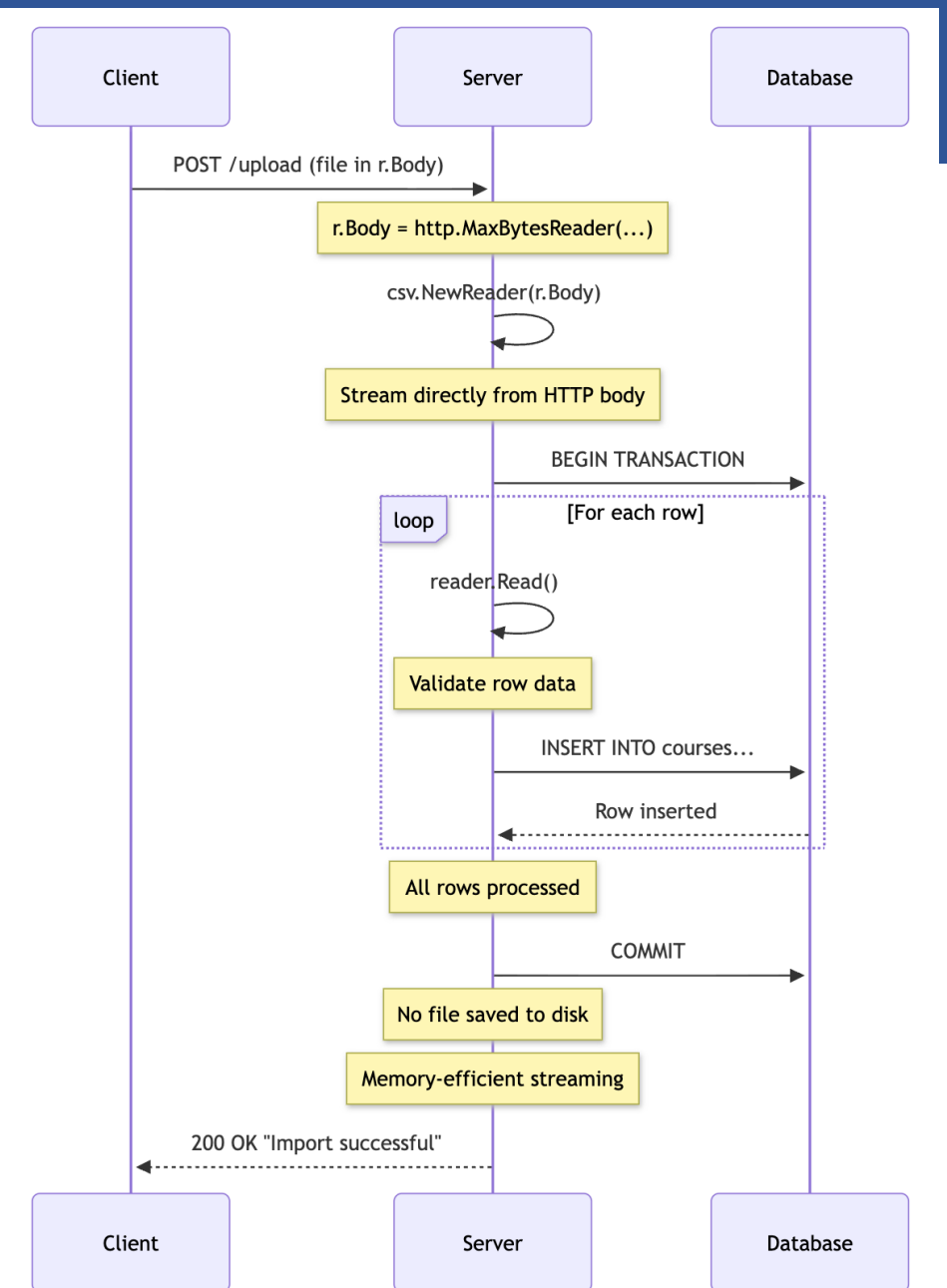
Questions:

- What if a CSV with 1000 rows has been inserted at 900 rows and then failed at row: 901
- What if there are 100 users upload 200MB CSV files



Alternatives ?

- Stream through backend

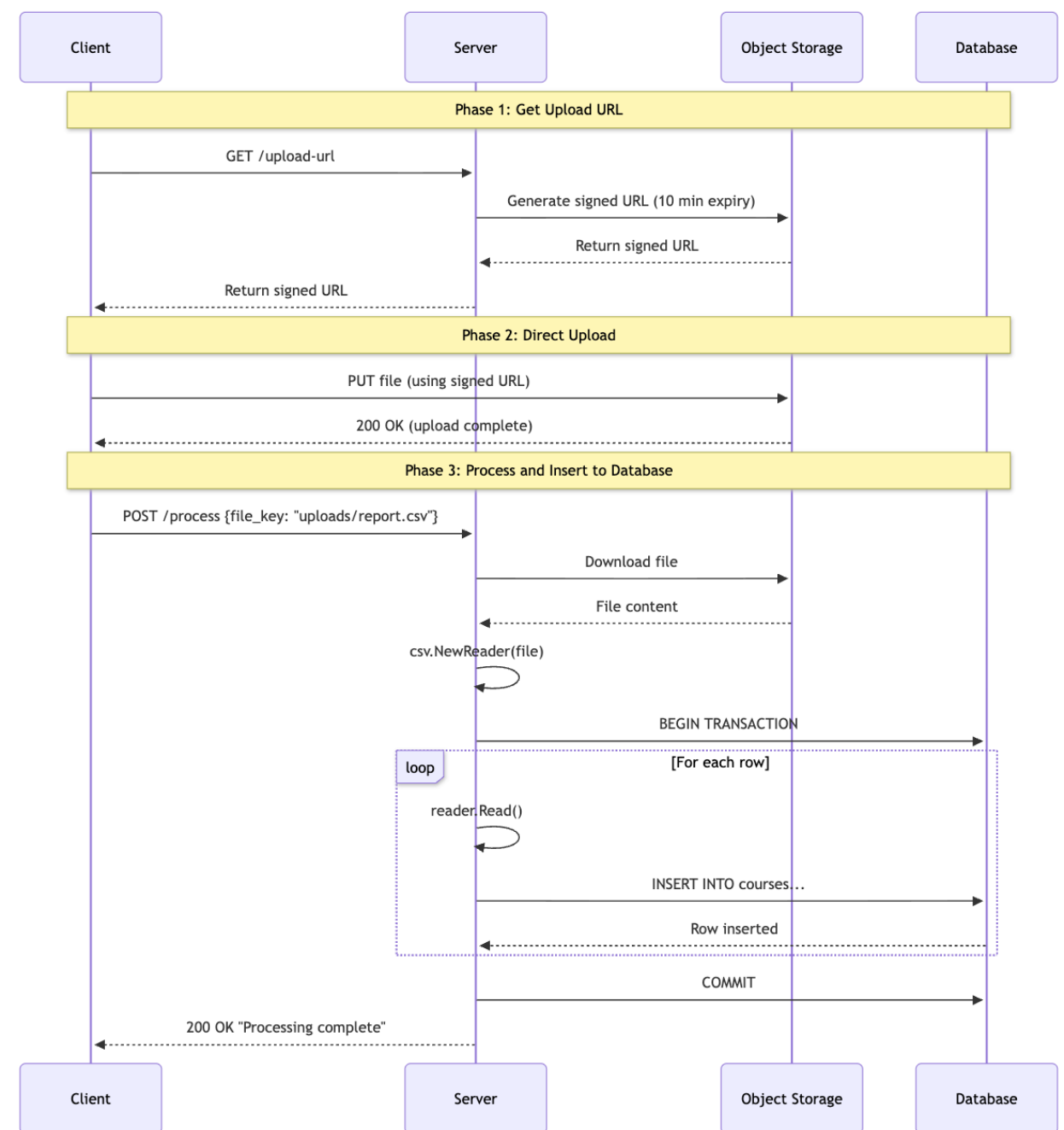


Alternatives ?

- Off-load the upload files to another server (object storage)

Improvement

- File parsing can be done in background (not in the main flow)



Object Storage

It is a computer data storage that manage data as objects, as opposed to file system, data blocks, or records

Common flows for Upload

- WebApp requests to Object Storage for Signed URL to upload a file

- Client uploads the file via the Signed URL

- Signed URL is only valid for a short of time

Flow Download

- WebApp requests to Object Storage for Signed URL to download a file

- Client shows/downloads the file via the signed URL

Upload Using Signed URL

Generating Signed URL:

```
import (
    "context"
    "time"
    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/service/s3"
)

func createUploadURL(
    ctx context.Context,
    client *s3.Client,
    bucket, key string,
) (string, error) {
    presigner := s3.NewPresignClient(client)
    req, err := presigner.PresignPutObject(
        ctx,
        &s3.PutObjectInput{
            Bucket:      aws.String(bucket),
            Key:        aws.String(key),
            ContentType: aws.String("application/pdf"),
        },
        func(o *s3.PresignOptions) {
            o.Expires = 10 * time.Minute
        },
    )
    if err != nil {
        return "", err
    }

    return req.URL, nil
}
```

The browser uploads directly:

```
await fetch(uploadURL, {
    method: "PUT",
    headers: {
        "Content-Type": "application/pdf ",
    },
    body: file
});
```

Download Using Signed URL

Generating Signed URL:

```
import (
    "context"
    "time"
    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/service/s3"
)

func createDownloadURL(
    ctx context.Context,
    client *s3.Client,
    bucket, key string,
) (string, error) {
    presigner := s3.NewPresignClient(client)
    req, err := presigner.PresignGetObject(
        ctx,
        &s3.GetObjectInput{
            Bucket: aws.String(bucket),
            Key:   aws.String(key),
        },
        func(o *s3.PresignOptions) {
            o.Expires = 10 * time.Minute
        },
    )
    if err != nil {
        return "", err
    }

    return req.URL, nil
}
```

The browser can then download directly:

```
window.location.href =
downloadURL;
```

Or it can download through fetch API:

```
const response = await
fetch(downloadURL);
```

Authentication & Authorization

Authentication and authorization

Who is using the application? What may they do?

- Registration and using Passwords
- Sessions and tokens
- JWT, OAuth 2.0, and OpenID Connect

continuing from State Handling

Aspect	Authentication (AuthN)	Authorization (AuthZ)
Definition	Prove the claim (identity)	Provision permission
Question	Who are you?	What are you allowed to do?
Purpose	Verify identity	Check permissions
When	Happens once (at login)	Happens on every protected request
Example	"You are student 123"	"Student 123 can view their own grades"
Failure Response	401 Unauthorized	403 Forbidden
Mechanism	Passwords, OIDC, biometrics	Access control lists, ownership checks

Basic password verification

Registration

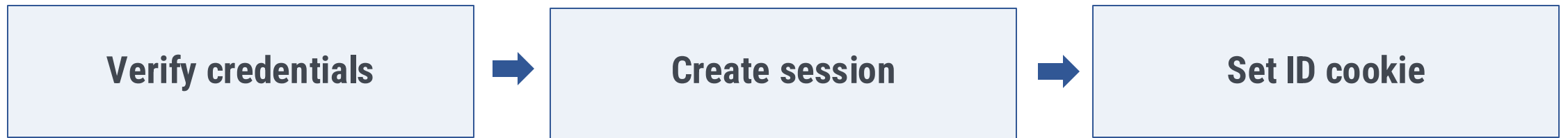
- Generate a unique salt.
- Compute an adaptive password hash (Argon2id, scrypt, PBKDF2).
- Store the hash and its parameters.

Login

- Load the stored hash.
- Verify the supplied password.
- Create a session only on success.

Use a maintained password-hashing implementation. Never store plaintext.

Login with a server session



Later request: cookie identifies the session.
Server loads the user and checks access to the requested resource.

The browser keeps continuity through the session-ID cookie.

Authentication, Authorization, Session, and Token

Authentication

Establish the user's identity.

Example:

The signed-in user is student 123.

Server-side session

State maintained across requests.

Example:

User 123 is signed in until expiry.

A signed-in user can still receive an access-denied response.

Authorization

Check access to this action.

Example:

Student 123 may view their own grades.

Token

A value the client presents.

Example:

A random ID identifying that session.

An opaque token can identifies server-side state

Cookie: session_id=a8f3c9...

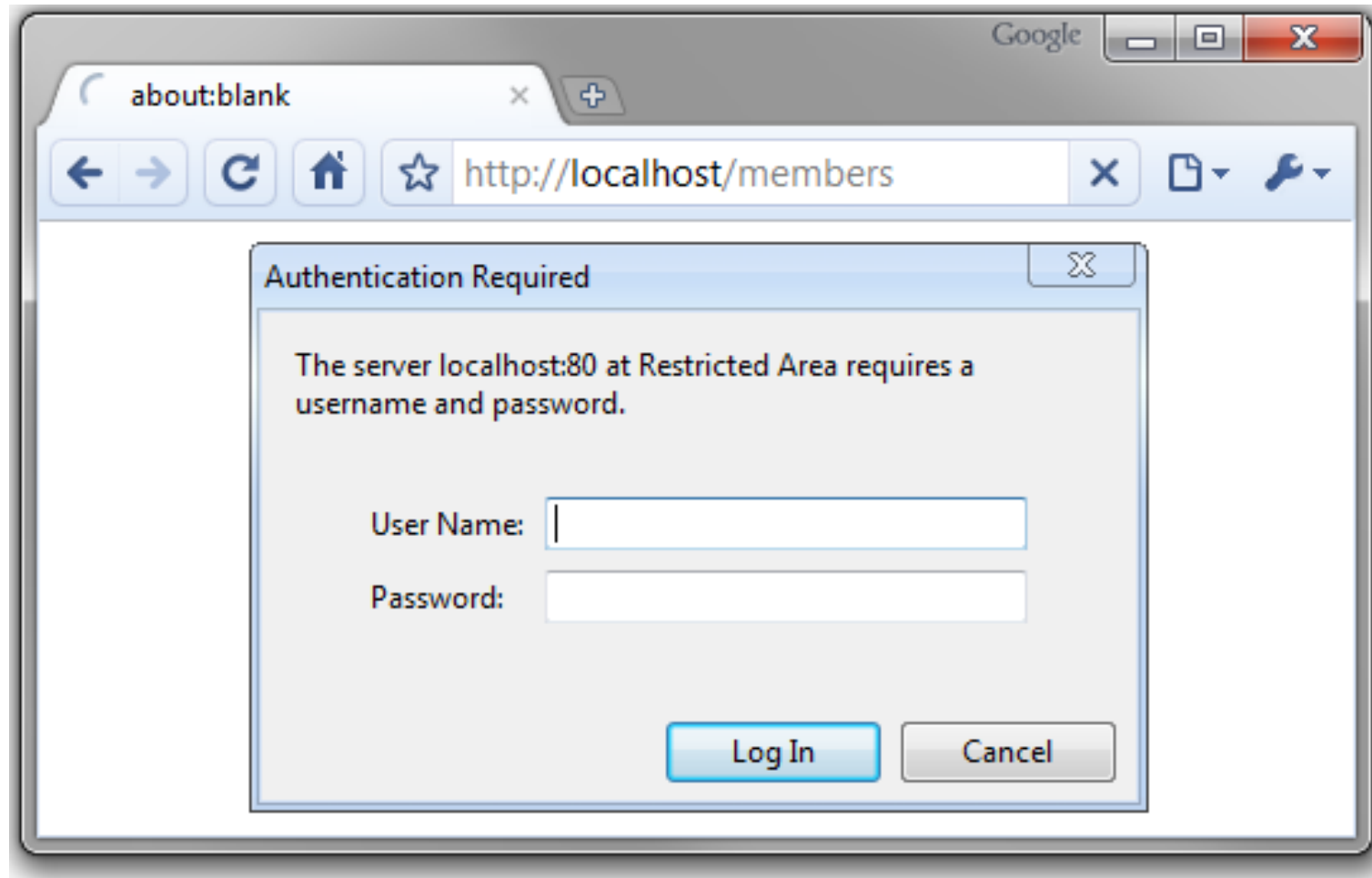
**Browser presents
A8f3c9...**



**Server lookup
userID: 123
expires: ...**

The server record supplies the meaning. The client holds an identifier.

Basic Auth



Basic Auth Implementation

```
GET /members  
Authorization: Basic <username>:<password>
```

```
func protectedHandler(w http.ResponseWriter, r *http.Request) {  
    username, password, ok := r.BasicAuth()  
  
    if !ok || !checkCredential(username,password) {  
        w.Header().Set(  
            "WWW-Authenticate", `Basic realm="restricted"`)  
  
        http.Error(w, "Unauthorized", http.StatusUnauthorized)  
        return  
    }  
  
    fmt.Fprintln(w, "Welcome!")  
}
```

Bearer tokens

```
GET /api/profile  
Authorization: Bearer <access-token>
```

```
const response = await fetch("/api/profile", {  
  headers: { Authorization: `Bearer ${accessToken}` }  
});
```

Possession matters. Keep tokens out of URLs, logs, and source code.

Authorization in a Go handler

```
// sess comes from the validated server session.  
// grade comes from the server's data store.  
if grade.StudentID != sess.UserID {  
    http.Error(w, "Forbidden", http.StatusForbidden)  
    return  
}  
  
// Now render the authorized grade record.
```

Check ownership on the server for every protected resource.

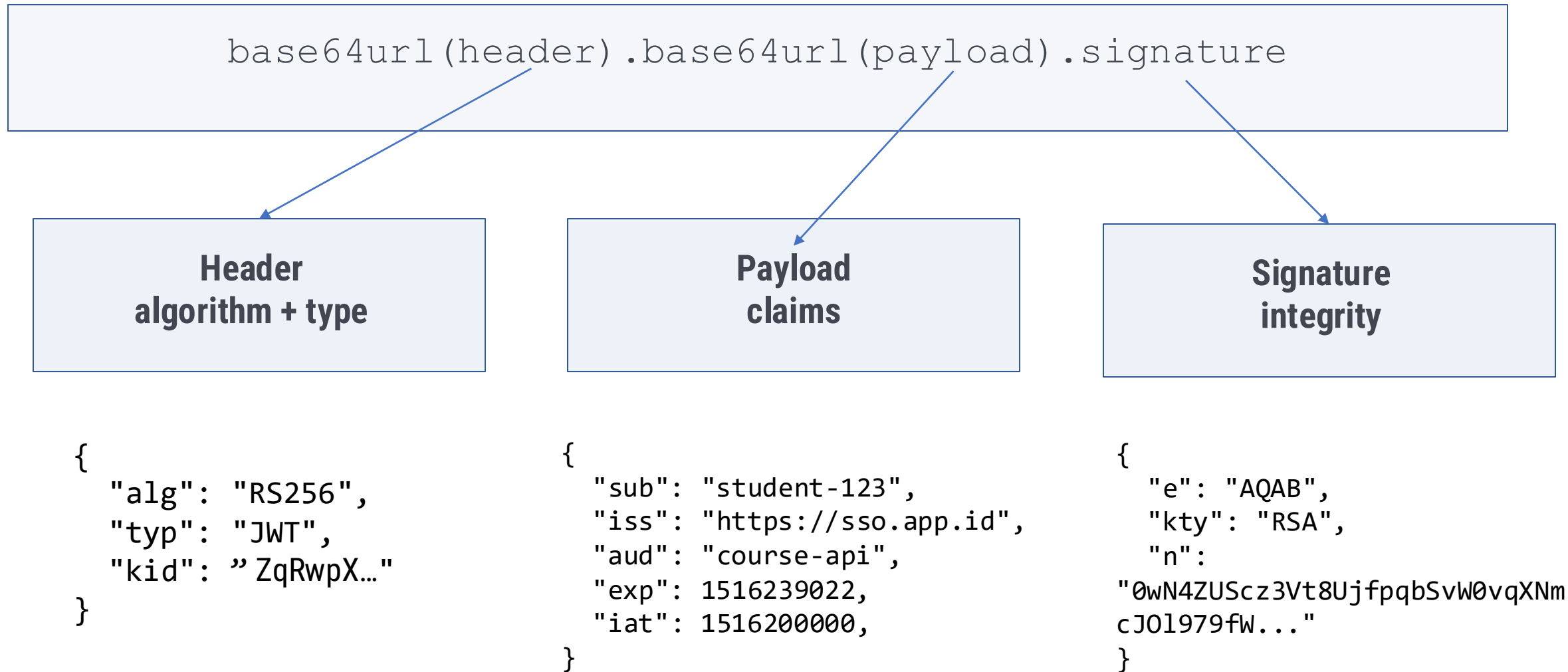
A token does not have to be in a JWT (JSON Web Token)

Representation	What the client holds	How the server checks it
Opaque token	Random identifier	Lookup or introspection
Signed JWT	Encoded claims + signature	Signature and required claims
Other formats	Protocol-specific representation	Rules defined by that protocol

OAuth access token: may be opaque or JWT.

OpenID Connect ID token: uses JWT format.

JWT Structure



Decoding and validating a JWT

Decoding the JWT

- Read the encoded header and claims.
- Anyone holding a signed JWT can usually do this.

Validate the JWT

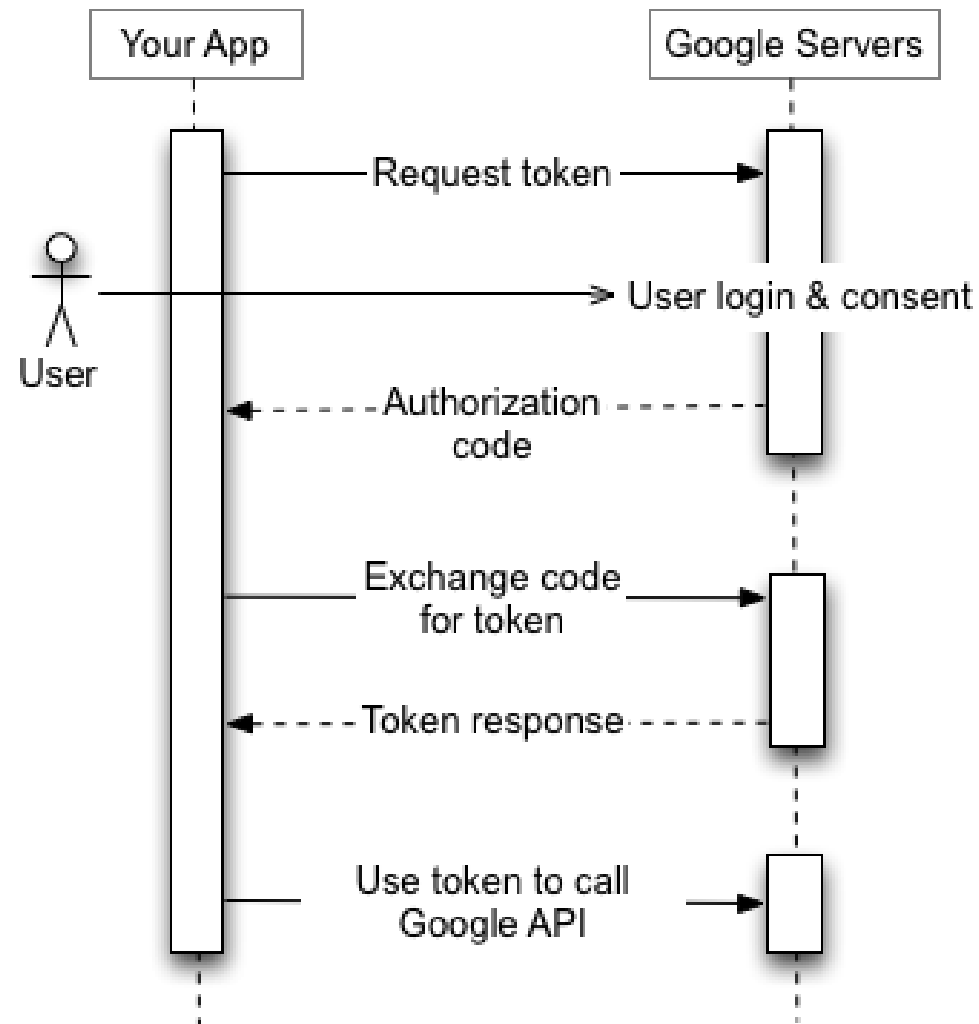
- Check signature with a trusted key.
- Require an allowed algorithm.
- Check issuer, audience, and expiry.
- Apply the correct token profile.

Use a maintained validator. Parsing JSON does not authenticate a user.

OAuth - Features

- A third party app can access user's data stored at service provider without requiring username and password.
- Delegated authorization protocol
- Explicit user consent is mandatory.
- Light-weight*
- Use Case
 - Website X can access your protected data at API Y
 - All without sharing your password off-site
 - especially when there isn't one like with OpenID
- Approach
 - Signed HTTP Requests
 - Safe, Password-less Token Exchange

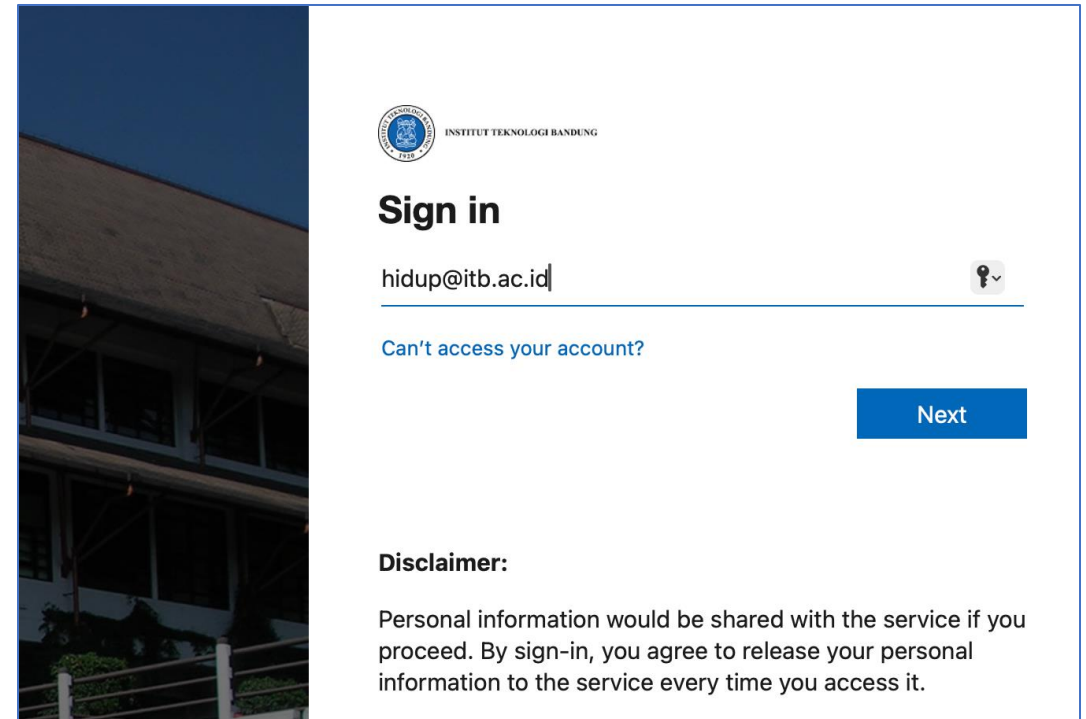
OAuth – An Overview



OpenID Connect: login through a provider

“Sign in using my campus account.”

The provider authenticates the student.
The application receives an ID token describing that authentication.
The application validates it and establishes its own login session.



 INSTITUT TEKNOLOGI BANDUNG

Sign in

hidup@itb.ac.id 

[Can't access your account?](#)

[Next](#)

Disclaimer:

Personal information would be shared with the service if you proceed. By sign-in, you agree to release your personal information to the service every time you access it.

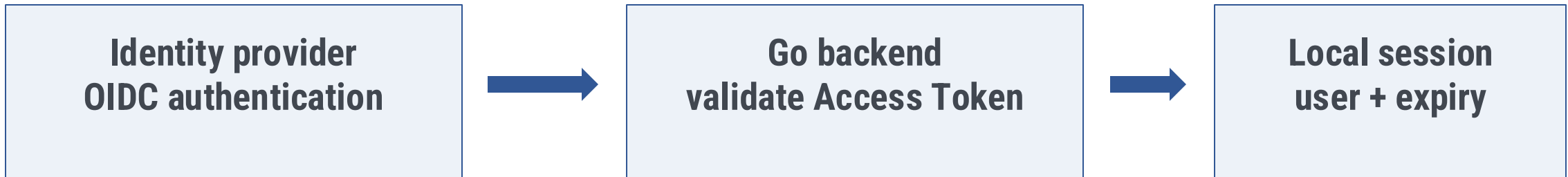
OAuth delegates access. OpenID Connect standardizes identity information.

OpenID Connect (OIDC) provides ID Token, access, and refresh tokens

Token	Consumer	Purpose
ID token	Client application	Evidence about user authentication
Access token	Resource server / API	Access within granted permissions
Refresh token	Authorization server	Obtain replacement access tokens

An ID token is not a substitute for an API access token.

OIDC Flow – An Overview with Go Server Side Script



Browser receives an HttpOnly session-ID cookie.
Go can retain provider tokens for server-to-server API calls.

JWT, OIDC, and a server-side session can all appear in the same login flow.